International University – Vietnam National University

Final Project

Design and Simulation of a Basic RISC-V RV32I Single-Cycle Processor in Verilog

Student: Pham Tran Dang Minh
Class: Digital System Design
Instructor: Vo Minh Thanh

Abstract

This report presents the design and simulation of a basic RISC-V RV32I single-cycle processor implemented in Verilog. The processor is designed to execute a subset of RV32I instructions, including arithmetic, logical, memory access, and branch operations. The system consists of several core modules such as the program counter, instruction memory, register file, immediate generator, control unit, ALU, data memory, multiplexers, and next-PC logic. A testbench was developed to verify the processor using a small instruction sequence. Simulation results show that the processor correctly executes the supported instructions, updates registers properly, handles memory operations, and performs branch decisions correctly. This project helps illustrate the fundamental concepts of datapath and control design in computer architecture.

Contents

Figure List

Table List

1. Introduction

RISC-V is an open-standard instruction set architecture that has become widely used in education and research because of its simplicity and modular structure. Among different processor implementations, the single-cycle processor is one of the most straightforward architectures for understanding how instructions are fetched, decoded, executed, and completed.

The main objective of this project is to design a basic RISC-V RV32I single-cycle processor in Verilog and verify its functionality through simulation. The design focuses on the essential datapath components and control logic needed to execute a subset of instructions. By implementing this processor, the project demonstrates how instruction fields are decoded, how

control signals are generated, how immediate values are formed, and how data flows through the processor.

2. Processor Architecture

The implemented processor follows a single-cycle architecture, which means that each instruction is completed in one clock cycle. In this architecture, all major operations such as instruction fetch, decode, execution, memory access, and write-back occur within the same cycle.

The processor consists of the following major components:

- Program Counter
- Instruction Memory
- Register File
- Control Unit
- Immediate Generator
- ALU Control Unit
- Arithmetic Logic Unit
- Data Memory
- Multiplexers
- Next-PC Logic
- Top-Level Integration Module

The overall execution flow is as follows. First, the program counter provides the address for instruction memory. The fetched instruction is then decoded into its fields, including opcode, source registers, destination register, funct3, and funct7. Based on the opcode, the control unit generates the necessary control signals. The register file reads source operands, while the immediate generator extracts and sign-extends the immediate value if required. The ALU performs arithmetic or logic operations, and data memory is accessed for load and store instructions. Finally, the result is written back to the register file, and the next PC value is determined for the next instruction.

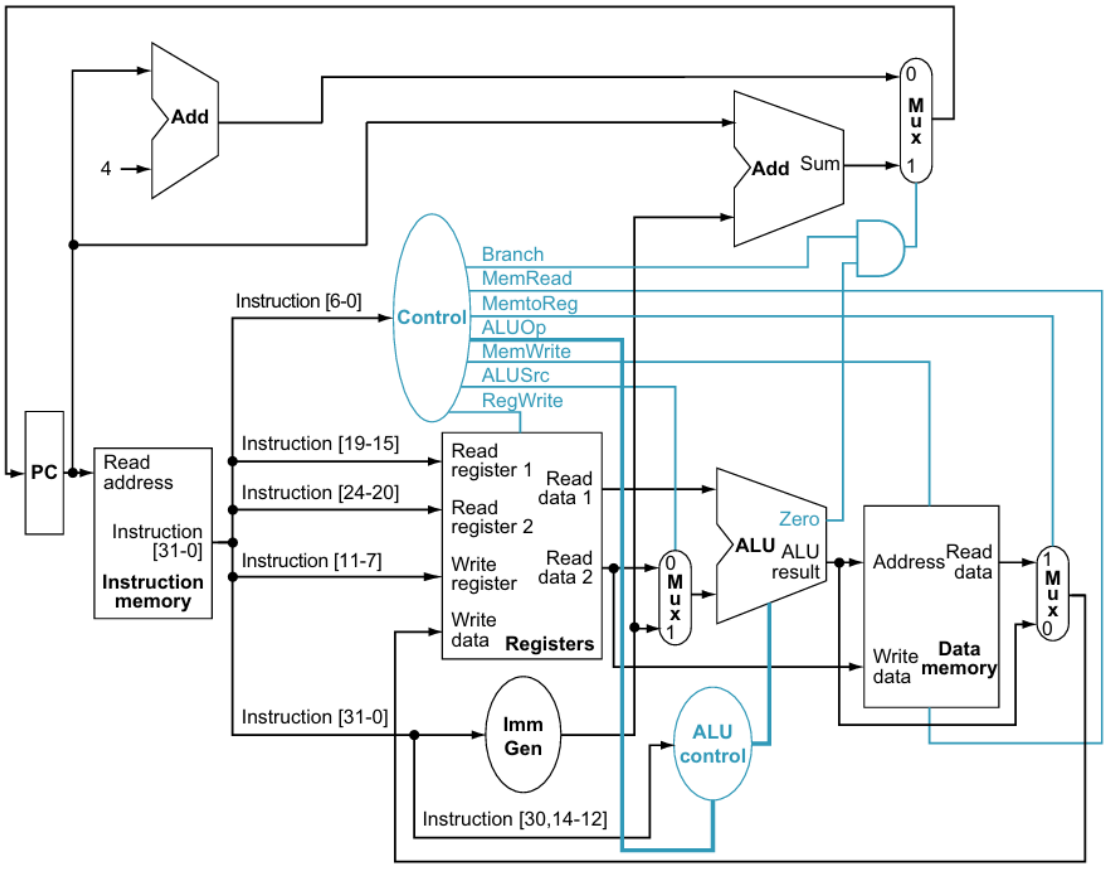

Figure 1.Datapath for the core single-cycle RISCV

3. Module Description

3.1 Program Counter

The program counter stores the address of the current instruction. On every positive clock edge, the PC is updated with the next PC value. When reset is active, the PC is cleared to zero. This ensures that instruction execution starts from address 0.

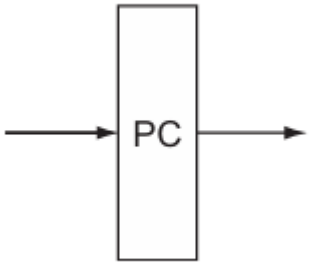

Figure 2.Program Counter

3.2 Instruction Memory

Instruction memory stores the machine-code instructions. The current PC value is used as the read address, and the instruction corresponding to that address is sent to the datapath. Since each instruction is 32 bits wide, word alignment is used by selecting address bits [7:2] for indexing the memory array.

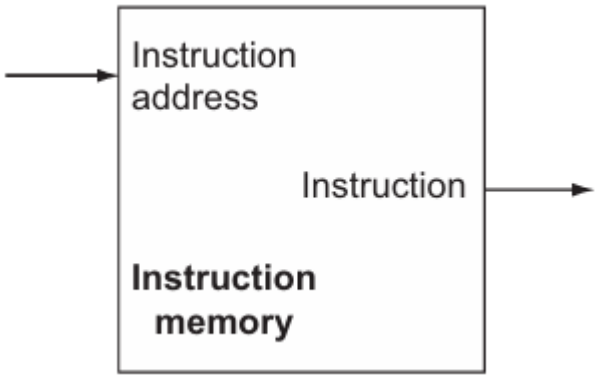

Figure 3.Instruction Memory

3.3 Register File

The register file contains 32 registers, each 32 bits wide. It supports two read ports and one write port. The source registers rs1 and rs2 are read combinationally, while the destination register rd is written on the positive clock edge when RegWrite is enabled. Register x0 is fixed to zero and cannot be modified.

Figure 4.Registers

3.4 Control Unit

The control unit decodes the opcode field of the instruction and generates the control signals required by the datapath. These control signals include branch control, memory read and write control, ALU source selection, register write enable, memory-to-register selection, ALU operation type, and immediate source type.

3.5 Immediate Generator

The immediate generator extracts immediate fields from the instruction and sign-extends them to 32 bits. In this project, the immediate generator supports the following instruction formats:

- I-type immediate
- S-type immediate
- B-type immediate

This module is essential for instructions such as addi, lw, sw, and beq.

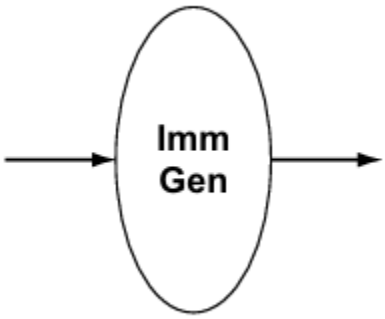

Figure 5.Immediate Generation Unit

3.6 ALU Control

The ALU control unit takes ALU_op, funct3, and funct7 as inputs and determines the exact ALU operation to be performed. For example, it distinguishes between add and sub, and it selects operations such as AND and OR when needed.

3.7 Arithmetic Logic Unit

The ALU performs arithmetic and logic operations on two operands. In this design, the ALU supports:

- Addition
- Subtraction
- Bitwise AND
- Bitwise OR

It also produces a zero output used for branch decisions.

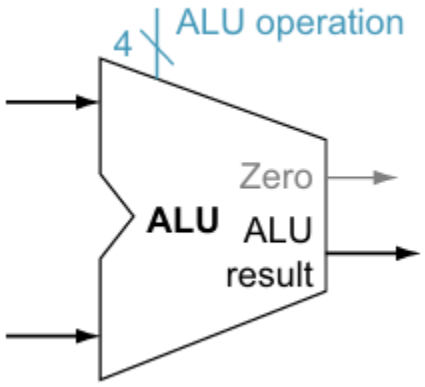

Figure 6.ALU

3.8 Data Memory

The data memory module is used for load and store instructions. For store operations, the ALU result is used as the memory address and the second register operand is written into memory. For load operations, the memory content at the calculated address is read and returned to the datapath.

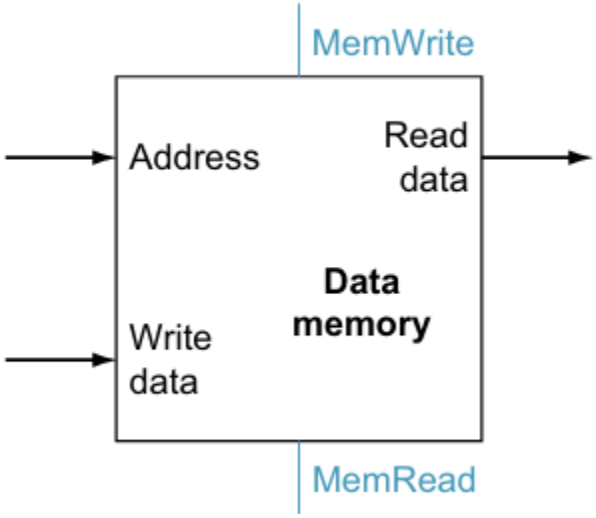

Figure 7. Data Memory

3.9 Multiplexers

Two 2-to-1 multiplexers are used in the design. One multiplexer selects the second ALU operand between register data and immediate data. The other multiplexer selects the write-back data between the ALU result and memory read data.

3.10 Next-PC Logic

The next-PC logic computes the next instruction address. Normally, the PC is incremented by 4. If a branch condition is true, the next PC becomes PC + immediate. In this project, branch decisions are based on the branch control signal and the ALU zero flag.

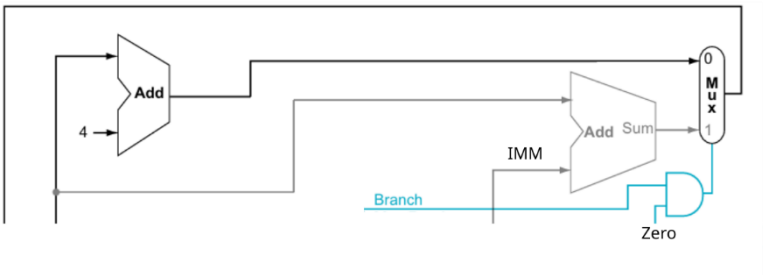

Figure 8.PC_next logic

3.11 Top Module

The top module connects all datapath and control components together. It acts as the complete processor system and ensures correct signal flow among all submodules.

4. Supported Instruction Set

The current processor supports a subset of the RV32I instruction set.

Name (Field size)	Field 7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format

Figure 9.Instructions of RISCV

4.1 R-type Instructions

The processor supports the following R-type instructions:

- add
- sub
- and
- or

These instructions operate on two source registers and store the result in one destination register.

4.2 I-type Instructions

The processor supports the following I-type instructions:

- addi
- andi
- ori

These instructions use one source register and one immediate operand.

4.3 Memory Instructions

The processor supports:

- lw
- sw

These instructions are used to read from and write to data memory.

4.4 Branch Instruction

The processor supports:

- beq

This instruction compares two registers and branches to a target address if they are equal.

5. Control Signal Design

The control unit generates different control signals depending on the opcode of the instruction. The signals used in this design are:

- branch
- MemRead
- MemToReg
- MemWrite
- ALUsrc
- RegWrite
- ALU_op
- ImmSrc

Table 1. The control signal of each instruction type

Instruction type	**RegWrite**	**ALUSrc**	**MemRead**	**MemWrite**	**MemToReg**	**Branch**	**ALU_op**	**ImmSrc**
R-type	1	0	0	0	0	0	10	00
Load	1	1	1	0	1	0	00	00
Store	0	1	0	1	0	0	00	01
Branch	0	0	0	0	0	1	01	10
I-type	1	1	0	0	0	0	10	00

For R-type instructions, the processor writes the ALU result back to the register file. For load instructions, it reads from memory and writes the memory data into a register. For store instructions, it writes register data into memory. For branch instructions, it compares two registers and updates the PC when the branch condition is satisfied.

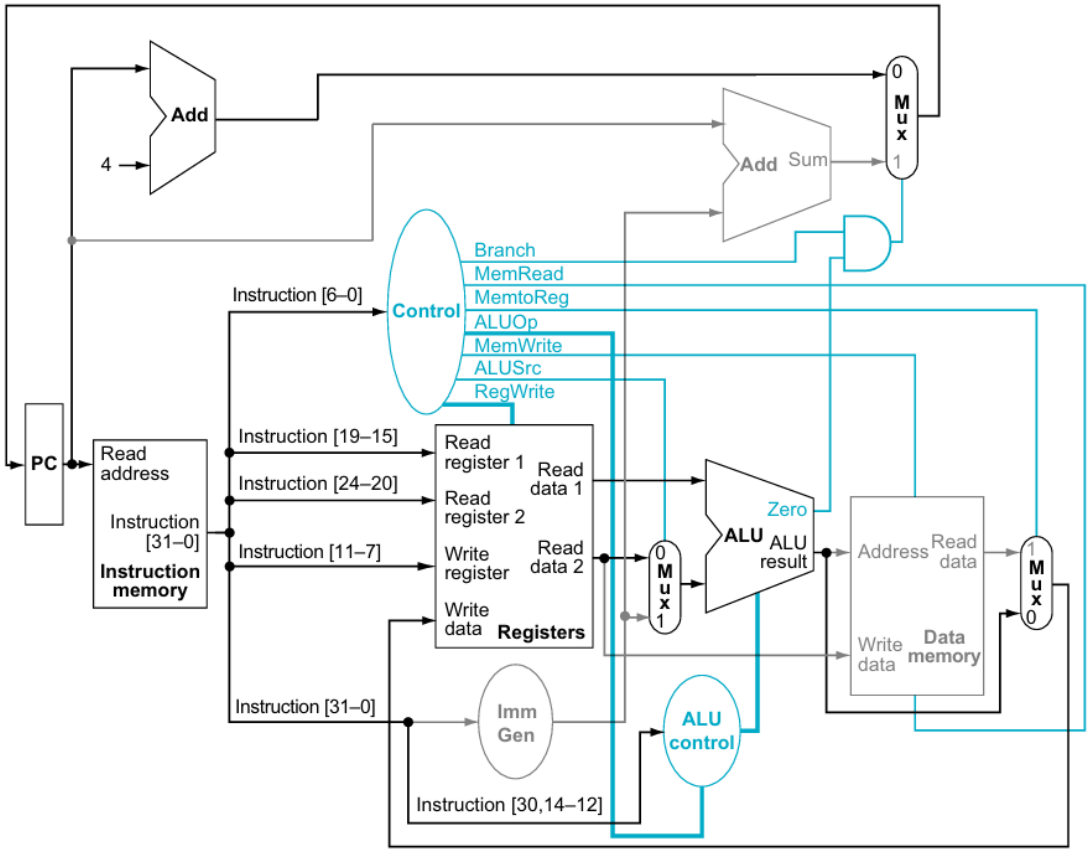

Figure 10.Datapath in operation for an R-type instruction

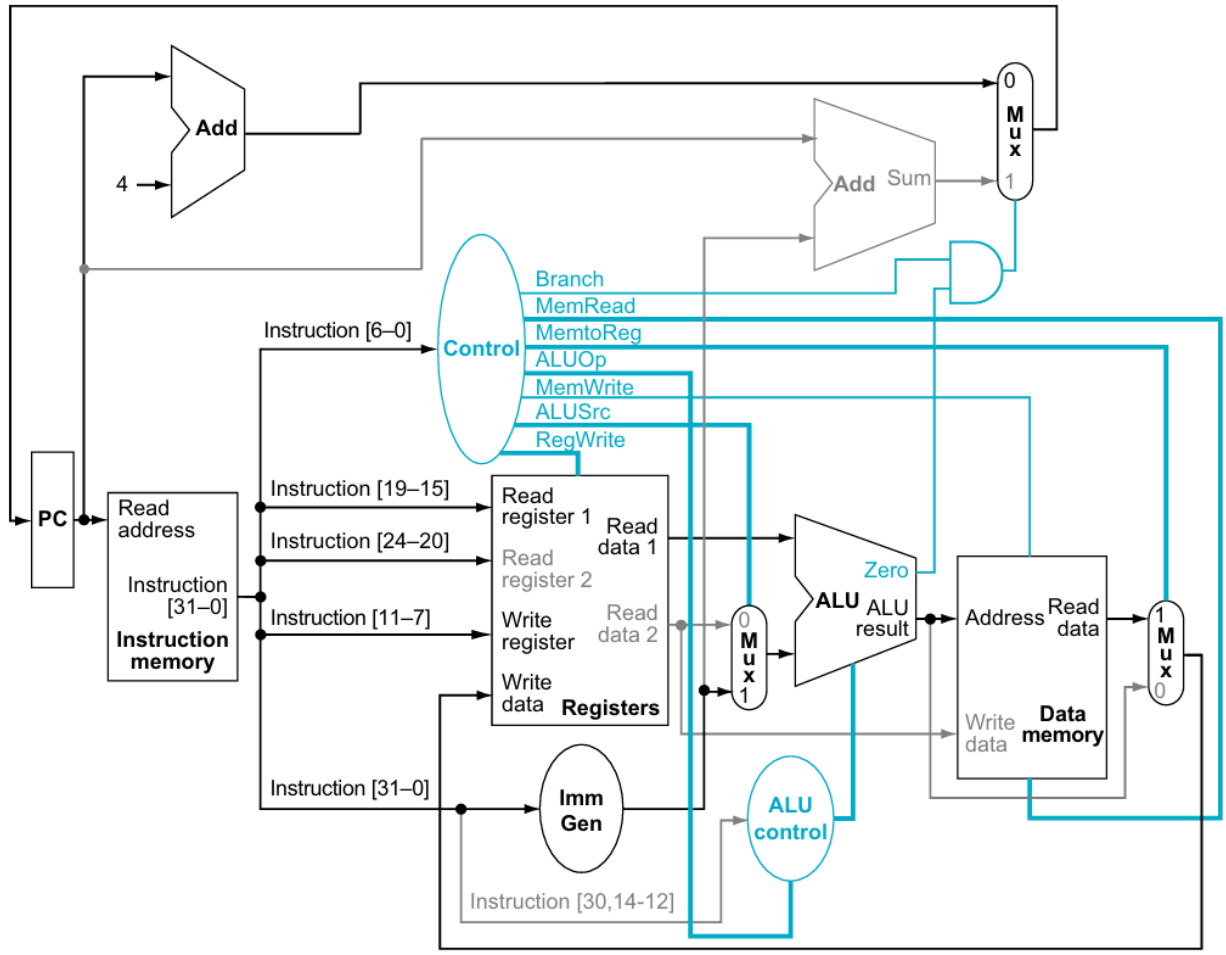

Figure 11.Datapath in operation for a load instruction

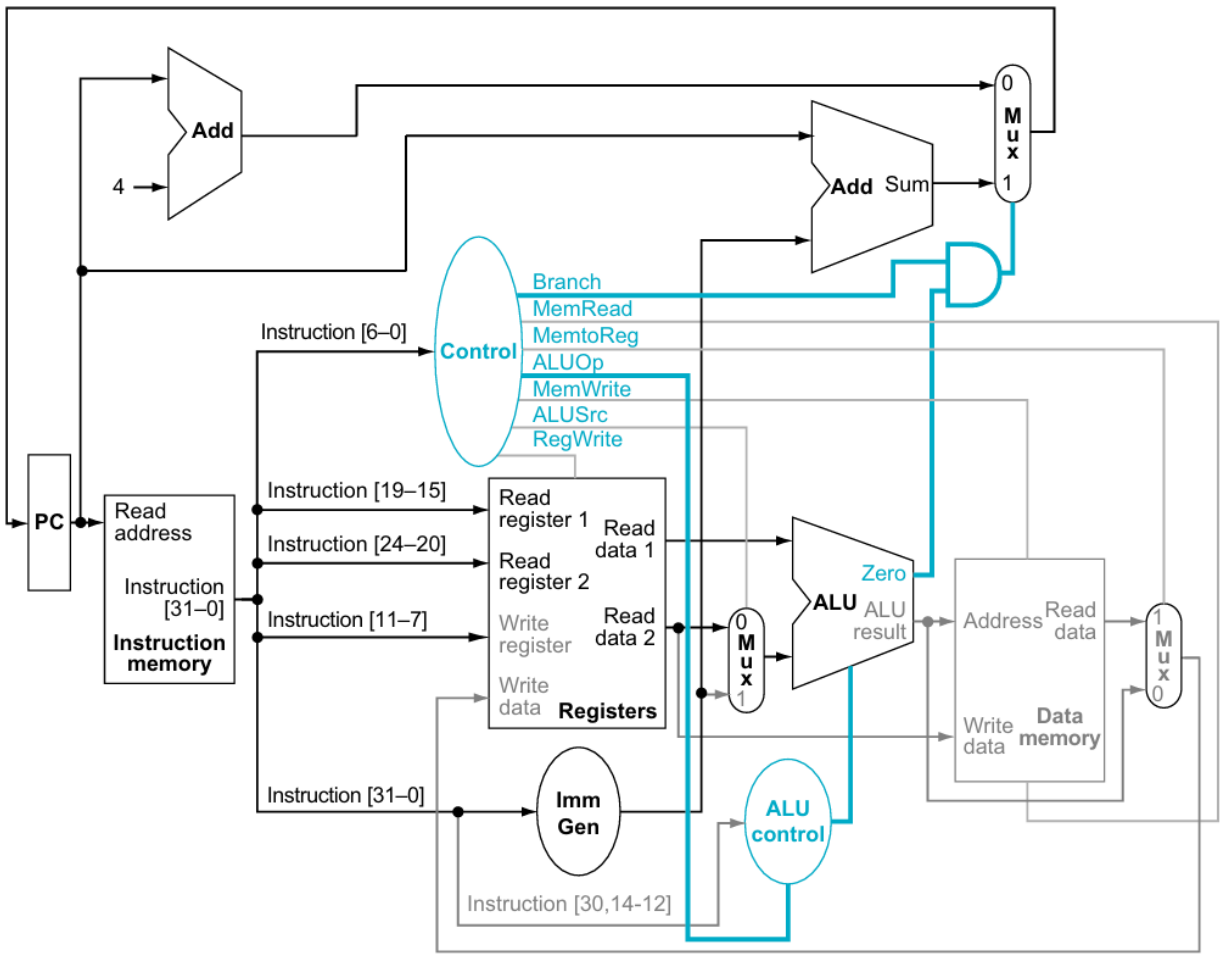

Figure 12.Datapath in operation for a branch-if-equal instruction.

6. Immediate Generation

Immediate generation is an important part of RISC-V instruction decoding because different instruction types store immediate values in different bit positions.

For I-type instructions, the immediate is taken from bits [31:20].

For S-type instructions, the immediate is formed by combining bits [31:25] and [11:7].

For B-type instructions, the immediate is formed by combining bits [31], [7], [30:25], [11:8], and an appended 0 as the least significant bit.

This design correctly sign-extends these immediate values to 32 bits before they are used in the datapath.

Table 2.Immediate formats used in the design

Instruction type	**Immediate field in instruction**	**Immediate construction**	**Description**
I-type	instr[31:20]	{{20{instr[31]}}, instr[31:20]}	Used for immediate arithmetic and load instructions such as addi and lw
S-type	instr[31:25] and instr[11:7]	{{20{instr[31]}}, instr[31:25], instr[11:7]}	Used for store instructions such as sw
B-type	instr[31], instr[7], instr[30:25], instr[11:8]	{{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}	Used for branch instructions such as beq

7. Testbench and Simulation

A Verilog testbench was created to verify the functionality of the processor. The testbench initializes the instruction memory with a small sequence of machine-code instructions and monitors the values of the PC and key registers during simulation.

The test program used in the simulation is shown below:

```
addi x1, x0, 5
addi x2, x0, 7
add  x3, x1, x2
sw   x3, 0(x0)
lw   x4, 0(x0)
beq  x3, x4, 8
addi x5, x0, 99
addi x6, x0, 42
```

The purpose of this program is to test arithmetic execution, memory operations, and branch behavior.

The expected behavior is as follows:

- x1 should contain 5
- x2 should contain 7
- x3 should contain 12

- Memory location 0 should store 12
- x4 should load the value 12 from memory
- beq should compare x3 and x4, detect equality, and branch
- x5 should remain 0 because the instruction is skipped
- x6 should contain 42

8. Simulation Result

The simulation results confirm that the processor executes the test program correctly. The arithmetic instructions produced the expected values in registers x1, x2, and x3. The store instruction correctly wrote the value 12 into data memory, and the load instruction correctly read it back into x4.

The branch instruction beq x3, x4, 8 also worked correctly. Since x3 and x4 both contained 12, the branch was taken, causing the processor to skip the instruction addi x5, x0, 99. As a result, x5 remained zero, while x6 was updated to 42 in the following instruction.

The final simulation results were:

- x1 = 5
- x2 = 7
- x3 = 12
- x4 = 12
- x5 = 0
- x6 = 42
- mem[0] = 12

These results demonstrate that the implemented datapath and control logic function correctly for the supported instructions.

9. Limitations

Although the processor works correctly for the tested instruction subset, it does not yet support the full RV32I instruction set. The following limitations remain:

- No support for jal and jalr
- No support for lui and auipc

- No support for branch types such as bne, blt, or bge
- No support for additional ALU operations such as XOR, shift, or set-less-than instructions
- Instruction memory is initialized manually in the testbench rather than using an external memory file

These limitations indicate that the current design is a simplified educational implementation rather than a complete RISC-V processor.

10. Conclusion

In conclusion, this project successfully implemented a basic RISC-V RV32I single-cycle processor in Verilog. The processor includes the main datapath components and control logic necessary to execute a subset of arithmetic, logical, memory, and branch instructions. The simulation results show that the processor performs correctly for the tested instruction sequence.

This project provides a practical understanding of processor design, including instruction decoding, control signal generation, immediate formation, ALU operation, memory access, and branch handling. It also serves as a strong foundation for future extensions such as full RV32I support or pipelined processor implementation.

11. Future Work

Future improvements to this project may include:

- Supporting the full RV32I instruction set
- Adding jump instructions such as jal and jalr
- Supporting more branch types
- Loading instructions from an external file using $readmemh
- Extending the design into a pipelined processor
- Adding more comprehensive test programs and waveform analysis

References

- (The Morgan Kaufmann Series in Computer Architecture and Design) David A. Patterson, John L. Hennessy - Computer Organization and Design RISC-V Edition_ The Hardware Software Interface-Morgan KaufmannCourse materials on Computer Architecture

- Materials on course Computer Architecture (Kiến Trúc Máy Tính (NXB Đại Học Quốc Gia 2019) – Instructor Phạm Quốc Cường) and Digital System Design (lectures of Instruct Vo Minh Thanh)